Journal of King Saud University – Computer and Information Sciences 35 (2023) 8–19

HOSTED BY

Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

Framework for automatic detection of anomalies in DevOps

Ahmed Hany Fawzy *, Khaled Wassif, Hanan Moussa

Faculty of Computers and Artificial Intelligence - Cairo University, Cairo, Egypt

ARTICLE INFO

Article history:
Received 30 October 2022
Revised 15 January 2023
Accepted 9 February 2023
Available online 15 February 2023

Keywords:
DevOps
AIOps
CI/CD
Anomaly detection
Release management
Machine learning

ABSTRACT

Log-based anomaly detection is important for improving the reliability and availability of software systems, especially those evolving using DevOps, owing to the huge number of logs generated during continuous practices. However, DevOps practitioners typically inspect and interpret the generated data and logs manually on specific occasions as part of the troubleshooting process. This process of manual inspection is time consuming and challenging because these data and logs have grown to an unmanageable size owing to the increasing size and complexity of the systems.

In this research paper, we introduce the **D**evOps **A**nomaly **D**etection **F**ramework (DADF). DADF is composed of two components that rely on Machine Learning (ML) and Artificial Intelligence (AI) to analyze the data and logs generated during DevOps practices to automatically detect anomalies. The first component is **A**nomaly **D**etection **B**efore **P**roduction (ADBP), which intends to detect anomalies in the prospective release before its operation in the production environment by adopting the Local Outlier Factor (LOF) technique on the data collected during implementation, building, testing, and deployment. The second component is **A**nomaly **D**etection **A**fter **S**taging (ADAS), which intends to detect anomalies after the operation of the released system by adopting the Vector Auto-Regression (VAR) technique on the data collected during monitoring from the system log, application log, and performance metrics (CPU and memory). We experimentally evaluated ADBP and ADAS in two different real-world industrial projects. The experimental results demonstrated that the accuracy, precision, recall, and F1-score of the ADBP component were 96%, 87.5%, 100%, and 93.3%, respectively, and the normalized Root Mean Squared Error (nRMSE) of the ADAS component was 2–19%. Hence, the results demonstrate the effectiveness of DADF in helping DevOps practitioners and researchers automatically detect anomalies throughout the lifecycle of DevOps by monitoring all DevOps' practices.

1. Introduction

Continuous delivery (CD) is an important conceptual part of DevOps' philosophy and practice, as it allows organizations to quickly deliver new features incrementally from concept to product. The purpose of CD is to ensure a constant flow of changes in production through an automated software production line (CD pipeline) that can have variable complexity depending on its phases and the tools that implement and support these phases (Rowse and Cohen, 2021; Shahin et al., 2017). Along with CD, continuous integration (CI) is a key concept in the DevOps approach. A typical example of CI is continuously integrating the changes made by the developer into the repository and building the code automatically. Automated testing begins when a code build succeeds. If automated tests pass, the changes are integrated into the code via the CI/CD pipeline and released into a production-like environment (Battina, 2019; Capizzi et al., 2019; Ebert et al., 2016).

Problem. Tools used to implement and support the CI/CD pipeline generate large amounts of data and logs in various formats (Capizzi et al., 2019). DevOps practitioners typically inspect and interpret generated data and logs manually on specific occasions as part of the troubleshooting process (Islam et al., 2021). However, this process of manual inspection is time-consuming and challenging because these data and logs have grown to an unmanageable

Abbreviations: DADF, DevOps Anomaly Detection Framework; ADBP, Anomaly Detection Before Production; ADAS, Anomaly Detection After Staging.

* Corresponding author.
E-mail address: ahmed.hany.fawzy@hotmail.com (A. Hany Fawzy).

Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksuci.2023.02.010
1319-1578/

size owing to the increasing size and complexity of the systems (Bosch and Bosch, 2020).

Solution. Thus, we propose a framework for the automatic detection of anomalies in DevOps by employing AIOps to address the aforementioned challenges using Machine Learning (ML) and Artificial Intelligence (AI) techniques and to address the challenges of building an AI/ML model for AIOps (Dang et al., 2019).

Contribution. This research paper introduces the **D**evOps **A**nomaly **D**etection **F**ramework (DADF) to help DevOps practitioners automatically detect anomalies throughout the DevOps lifecycle,

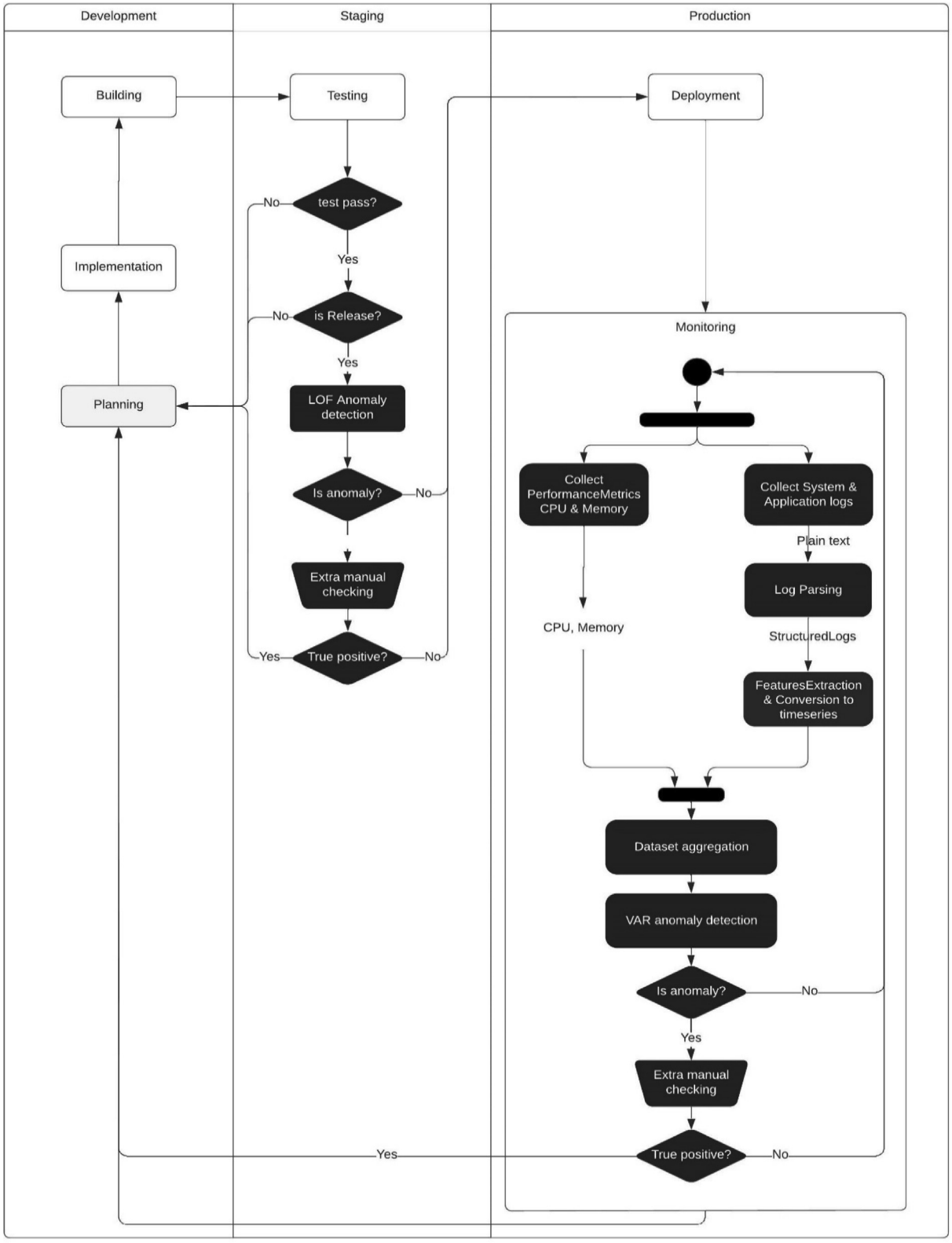

Fig. 1. **D**evOps **A**nomaly **D**etection **F**ramework (DADF). "Figures with white background illustrate the typical DevOps phases "Implementation, Building, Testing, Deployment and Monitoring", and figures with black background illustrate the steps of the proposed anomaly detection framework"

as shown in Fig. 1. On the one hand, this framework can automatically detect anomalies before deploying the system into the production environment by analyzing the data collected during continuous practices (development, testing, building, and deployment) to mitigate production problems. On the other hand, it detects anomalies during monitoring by establishing a baseline of the normal behavior of the monitored system by employing a multivariate time-series approach to predict the upcoming values and compare them with the actual ground truth values, instead of the traditional fixed rules and threshold approaches that are widely used to monitor software systems.

Organization. The remainder of this paper is organized as follows. Section 2 surveys the related work. Section 3 is divided into three subsections: Section 3.1 introduces an overview of the proposed framework and its components, as well as the deployment model used in this study. Sections 3.2 and 3.3 discuss the materials, methods, and experimental setup of the ADBP and ADAS components, respectively. Section 4 presents the results and discussion of both the ADBP and ADAS components in Sections 4.1 and 4.2, respectively, followed by the proposed anomaly detection framework in Section 4.3. Finally, Section 5 concludes the paper.

2. Literature review

Logs are widely used in practice for the maintenance and diagnosis of various software systems, owing to the abundant information they provide. Log-based anomaly detection has become a popular research topic (Capizzi et al., 2019; Du et al., 2017; Farshchi et al., 2018; He et al., 2018; Huch et al., 2018; Islam et al., 2021; Khatuya et al., 2018; Lou et al., 2010; Sun et al., 2016; Yang et al., 2021; Zhang et al., 2019). Several previous studies have employed ML and data-mining techniques to detect anomalies and identify problems (Cândido et al., 2021; Le and Zhang, 2022; Li et al., 2020). Log-based anomaly detection studies are categorized as supervised, unsupervised, or semi-supervised.

Supervised log-based anomaly detection is employed if a considerable amount of labeled training data is available. For example, (Farshchi et al., 2018; Sun et al., 2016) proposed a regression-based approach to detect anomalies in cloud systems by leveraging the correlation between the activity logs of DevOps rolling upgrade sporadic operation and the effect of the operation on cloud resources. (Huch et al., 2018) proposed machine learning classifiers to predict a system's health status and evaluated their approach in an industrial case study on a large real-world dataset using a predefined time interval "window" for long DevOps operations. (Khatuya et al., 2018) introduced ADELE, which is an empirical data-driven methodology for the early detection of anomalies in data storage systems. ADELE learns from the system's own history to establish a baseline of normal behavior and provides accurate indications of the time period when something is amiss for a system. (Zhang et al., 2019) introduced LogRobust to extract semantic information of log events and identify unstable log data, and then utilized an attention-based Bidirectional Long-Short-Term Memory (Bi-LSTM) model to detect anomalies. Although supervised learning approaches have superior accuracy, their usage in practical AIOps solutions is limited due to the difficulty of obtaining labeled data.

Unsupervised and semi-supervised log-based anomaly detection techniques are more practical than supervised techniques because they are less dependent on labeled training data. For example, (Lou et al., 2010) proposed Invariant Mining (IM) to extract the relationship between log events from log event count vectors. (Du et al., 2017) proposed Deeplog, which uses LSTM to predict the next log event and compare it with the current ground truth to detect anomalies. (He et al., 2018) proposed "Log3C", a clustering-based approach to identify problems of online service systems by utilizing both sequences of log events and system key performance indicators. (Capizzi et al., 2019) introduced the SVADS approach to detect anomalies in the DevOps CI/CD pipeline by comparing the current incoming release with previous releases based on predefined metrics. (Yang et al., 2021) proposed PLELog, which is a semi-supervised approach for learning historical anomalies through probabilistic label estimation. (Islam et al., 2021) implemented an automated monitoring system that evolved using DevOps and utilized Deep-learning Neural Networks (DNN) to simultaneously detect anomalies in near real-time in multiple platform components to overcome the reduction in efficiency. The difficulty of building high-accuracy unsupervised models lies in the complexity of the internal logic of services and the large volume of data that needs to be analyzed (Dang et al., 2019).

3. Material and methods

3.1. Overview

The work conducted in this research follows the **D**evelopment, **S**taging, and **P**roduction (DSP) model (Allen Jones, 2022; Bass, 2017). In this model, activities are categorized into three deployment environments.

- Development: An environment designated for developers to work on and test new features; once they are convinced that their code is ready for prime time, it is moved to staging.
- Staging: An environment similar to the production environment, whose main purpose is to test new features that are ready to be merged into the system to ensure that they do not break the existing production environment.
- Production: The environment in which a prospective release is deployed and released for utilization by end users.

The amount of data and logs generated during the development and staging activities could help detect anomalies early in the prospective release before its deployment into production. The data and logs generated during the runtime of the system in the production environment can also be utilized to detect anomalies and mitigate the possibility of system outages. Therefore, this research paper addressed these two scenarios.

3.2. *A*nomaly *d*etection *b*efore *p*roduction (ADBP)

Based on the unlabeled nature of the data generated during the implementation of DevOps practices in both development and staging environments, we employed the Local Outlier Factor (LOF) (Breunig et al., 2000), which is an unsupervised ML technique for computing the local density deviation of a multivariate data point compared to its neighbors. The ADBP model is built directly after finalizing the activities related to development, building, and testing practices when the prospective software release is announced to be ready for production. Therefore, just before moving the release to production, an extra anomaly detection step can be triggered to identify whether the prospective release deviates from the existing distributions in terms of the following metrics: number of lines of code per commit, number of failed tests compared to the total number of tests, number of failed builds compared to the total number of builds, and number of failed deliveries (deployments to a staging environment) compared to the total number of deliveries, as listed in Table 1, which contains the set of metrics and their derived features.

Experiment Setup: In order to build the ADBP component, first we developed a tool using the PHP–Laravel framework (Stauffer,

Table 1
Metrics and features of the ADBP component.

DevOps Stage	Metrics	Extracted Feature
Implementation	Number of lines of code and number of commits	Number of lines of code per commit
Testing	Number of tests failed and total number of tests	Number of failed tests compared to the total number of tests
Building	Number of failed builds and total number of builds	Number of failed builds compared to the total number of builds
Deployment (to the staging environment)	Number of failed deliveries and total number of deliveries	Number of failed deliveries compared to the total number of deliveries

2019) and MYSQL database (DuBois, 2008) in order to collect the aforementioned metrics since the last released software version via the REST APIs provided by GitLab ("GitLab," 2022), and extract the derived features listed in Table 1 to save them into the MYSQL database in order to construct the dataset, in which each entry represents a collection of values for all of the derived features. Second, we developed another tool using Python and exploited the Python library scikit-learn (Paper and Paper, 2020), which implements the LOF algorithm and utilized K-fold cross-validation to build the ADBP model. After training and building the ADBP model, the last entry in the dataset, which indicates the prospective release, is compared with its neighbors to check whether it is a normal release or an anomaly that deviates from its neighbors (previous distributions).

Before running the pipeline job that deploys the system into production, the anomaly detection job is triggered to run the first tool to collect the metrics of the prospective release and insert a new entry into the dataset. The dataset is then passed to the second tool to be normalized and preprocessed for use by the ADBP model. The ADBP model uses K-fold (10-fold) cross-validation to split the dataset into training and test sets, where the (K-1) fold is used for training and always leaves 1-fold for testing. Then, after building and training the ADBP model, the status of the prospective release is identified, and if no anomaly is detected, the deployment to the production job can be manually triggered via the GitLab CI/CD pipeline. However, if an anomaly is detected, the DevOps team will be notified, and additional manual checking should be performed to confirm that it is a true positive anomaly, as shown in Fig. 2, which illustrates the normal DevOps phases (elements with white backgrounds) with the tools used to support these phases, the metrics collected beside each phase, and the proposed steps of the ADBP component (elements with black backgrounds).

Case Study: The ADBP component was applied to an industrial marketplace web application developed by an Egyptian software house that supports Egyptian governmental digital transformation initiatives. This marketplace application was developed using DevOps practices and the following set of tools: PHP-Laravel Framework, MYSQL database, Docker (Anderson, 2015) deployed on the Debian operating system, GitLab for version control, and CI/CD pipeline.

3.3. *A*nomaly *d*etection *a*fter *s*taging (ADAS)

Nowadays, sophisticated and automated monitoring approaches are widely used to monitor the health of software systems during their operation. However, most existing monitoring approaches still rely on statistics and heuristics of observed health indicators based on resource utilization thresholds (Islam et al., 2021; Pourmajidi et al., 2018). Such traditional approaches require in-depth expertise and knowledge of the internals of the system to define and set monitoring rules and thresholds. Therefore, several studies have employed AI during software operations (AIOps) instead of fixed rules and thresholds to address the aforementioned challenges.

Experiment Setup: To detect anomalies during monitoring without relying on thresholds, we propose the ADAS component that is built using Python and its packages (Statsmodel, Pandas, and NumPy), and intended to construct a baseline of the system's behavior based on the history of the monitored system. The high-level steps of the process of building the ADAS component are shown in Fig. 3 and are discussed in detail in subsections [3.3.1–3.3.6].

Case Study: The ADAS component was applied to a governmental enterprise system serving Egyptian citizens, and is composed of two subsystems that interact with each other. Hence, an anomaly that occurs in one of them may affect the other, and this is a complex case that limits the applicability of traditional monitoring systems. The first subsystem was built using NodeJS, and the second subsystem was built using PHP-Laravel. Both subsystems were deployed in a Linux environment operated by the CentOS-7 operating system (Alibi and Roy, 2016).

3.3.1. Collect system performance metrics (CPU and Memory)

To collect the performance metrics, a combination of the Linux commands (AWK, Echo, Top, and Grep) was used to output the CPU and memory usage into an external file using the Crontab tool, which was configured to run every minute. While the Crontab tool offers various frequencies to run the performance metrics logging command, we chose a one-minute frequency to be more supportive of other solutions, as this is a common frequency that is used in many other monitoring tools, especially the widely used cloud monitoring tool AWS CloudWatch.

3.3.2. Collect the system and application logs

In this paper, various types of logs were collected to cover all components of the system. Hence, we collected operating system logs that were logged using the RSyslog utility (Fig. 4a) and application logs (Fig. 4b).

3.3.3. Logs parsing

The first and most crucial step in log analysis is log parsing, which is an automated process for converting raw text logs into a structured stream of events (He et al., 2016).

To achieve the goal of automated log parsing, we utilized the drain log parser as the parsing tool because it shows superior accuracy, efficiency, and robustness compared to other log parsers evaluated by (Zhu et al., 2019) on various types of logs.

3.3.4. Derivation of metrics (features extraction) and conversion to time-series

In contrast to log events, performance metrics such as CPU and memory usage are logged with a fixed frequency, usually a one-minute frequency, similar to that offered by the widely used cloud monitoring tool CloudWatch (Farshchi et al., 2018; Sun et al., 2016). Hence, performance metrics and logged events must be mapped.

Therefore, to facilitate the mapping process, we introduced a predefined fixed 5-min time interval (window), as introduced in (Huch et al., 2018), and extracted the count of log events per severity during each window. The introduction of the window enabled us to overcome the issue of working at different frequencies. In addition, for the performance metrics, we worked on the same predefined window size and derived new data metrics by calculating the average CPU and memory usage for each window. (Tables 2a, 2b, and 2c show samples of the derived features).

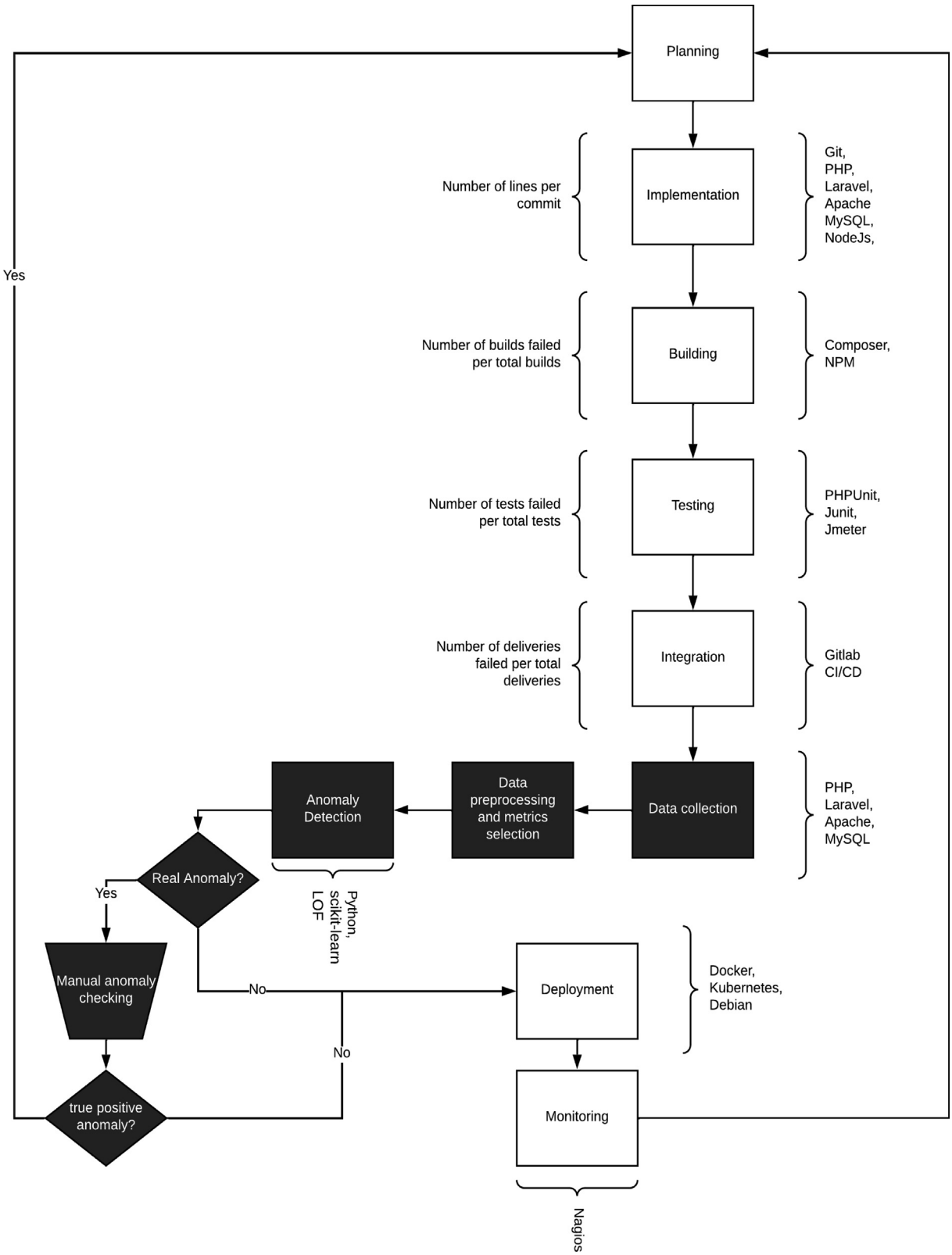

Fig. 2. ADBP Process and toolchain.

3.3.5. Aggregating and mapping the performance data and logs metrics

To avoid tying the ADAS component to a specific type of log or domain-specific context, we derived features that could be extracted from most types of logs and mapped them to performance metrics. Furthermore, after unifying the frequencies, metric aggregation was performed to construct the final dataset.

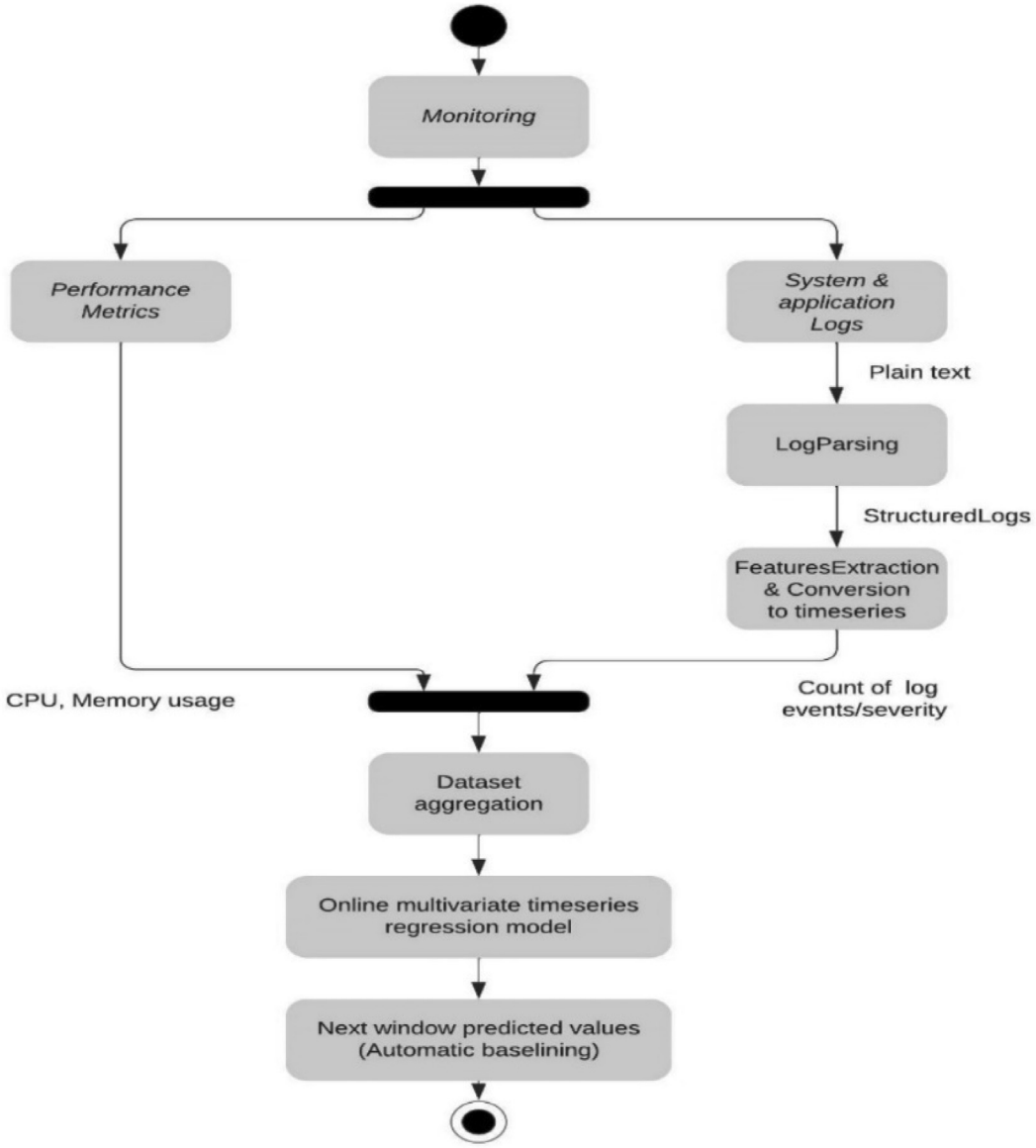

Fig. 3. ADAS process and steps.

```
Time: 2022-05-23 15:53:36, Facility: mail, Priority: info, Hostname: bckapp1, Message: 3113B649EFC: message-id=<20220523135336.311
Time: 2022-05-23 15:53:36, Facility: mail, Priority: info, Hostname: bckapp1, Message: 2F3EB649EF9: sender non-delivery notificati
Time: 2022-05-23 15:53:36, Facility: mail, Priority: info, Hostname: bckapp1, Message: 3113B649EFC: from=<>, size=2656, nrcpt=1 (q
Time: 2022-05-23 15:53:36, Facility: mail, Priority: info, Hostname: bckapp1, Message: 2F3EB649EF9: removed
Time: 2022-05-23 15:53:36, Facility: daemon, Priority: info, Hostname: bckapp1, Message: Removed slice User Slice of apache.
Time: 2022-05-23 15:53:36, Facility: mail, Priority: info, Hostname: bckapp1, Message: 3113B649EFC: to=<root@bckapp1.aptaa.local>,
Time: 2022-05-23 15:53:36, Facility: mail, Priority: info, Hostname: bckapp1, Message: 3113B649EFC: removed
Time: 2022-05-23 15:54:01, Facility: daemon, Priority: info, Hostname: bckapp1, Message: Created slice User Slice of apache.
Time: 2022-05-23 15:54:01, Facility: daemon, Priority: info, Hostname: bckapp1, Message: Started Session 419954 of user apache.
Time: 2022-05-23 15:54:01, Facility: daemon, Priority: info, Hostname: bckapp1, Message: Started Session 419953 of user apache.
Time: 2022-05-23 15:54:01, Facility: daemon, Priority: info, Hostname: bckapp1, Message: Started Session 419956 of user apache.
Time: 2022-05-23 15:54:01, Facility: daemon, Priority: info, Hostname: bckapp1, Message: Started Session 419955 of user apache.
Time: 2022-05-23 15:54:01, Facility: daemon, Priority: info, Hostname: bckapp1, Message: Started Session 419957 of user root.
Time: 2022-05-23 15:54:01, Facility: cron, Priority: info, Hostname: bckapp1, Message: (apache) CMD (/scripts/getfiles.sh)
Time: 2022-05-23 15:54:01, Facility: cron, Priority: info, Hostname: bckapp1, Message: (apache) CMD (/scripts/extractfiles_im.sh)
Time: 2022-05-23 15:54:01, Facility: cron, Priority: info, Hostname: bckapp1, Message: (apache) CMD (/scripts/downloader.sh)
Time: 2022-05-23 15:54:01, Facility: cron, Priority: info, Hostname: bckapp1, Message: (apache) CMD (/scripts/extractfiles.sh)
Time: 2022-05-23 15:54:01, Facility: cron, Priority: info, Hostname: bckapp1, Message: (root) CMD (/var/scripts/aa.sh^I)
```

Fig. 4a. CentOS7 RSyslog.

```
[2022-05-23 15:53:36] local.INFO: array (
  0 =>
  array (
    'total' => '256.0',
  ),
)
[2022-05-23 15:53:56] local.INFO: ******************************
[2022-05-23 15:53:56] local.ERROR: ErrorException: Undefined index: pda_id in /var/www/html
Stack trace:
#0 /var/www/html/alex-portal/app/Http/Controllers/IMController.php(203): Illuminate\Foundat
#1 [internal function]: App\Http\Controllers\IMController->handlePdaTransactions(Object(Ill
#2 /var/www/html/alex-portal/vendor/laravel/framework/src/Illuminate/Routing/Controller.php
#3 /var/www/html/alex-portal/vendor/laravel/framework/src/Illuminate/Routing/ControllerDisp
#4 /var/www/html/alex-portal/vendor/laravel/framework/src/Illuminate/Routing/Route.php(219)
#5 /var/www/html/alex-portal/vendor/laravel/framework/src/Illuminate/Routing/Route.php(176)
#6 /var/www/html/alex-portal/vendor/laravel/framework/src/Illuminate/Routing/Router.php(680
#7 /var/www/html/alex-portal/vendor/laravel/framework/src/Illuminate/Routing/Pipeline.php(3
#8 /var/www/html/alex-portal/vendor/laravel/framework/src/Illuminate/Routing/Middleware/Thr
```

Fig. 4b. Laravel application log.

Table 2a
Sample of CPU, Memory derived features.

Time	CPU	Memory
5/23/2022 15:50	28.633333	8.2
5/23/2022 15:55	27.46	8.28
5/23/2022 16:00	32.02	8.26

Table 2b
Sample of application log derived features.

Time	App_error	App_info
5/23/2022 15:50	4	5
5/23/2022 15:55	2	3
5/23/2022 16:00	1	2

Table 2c
Sample of operating system derived features.

Time	rsys_info	rsys_notice
5/23/2022 15:50	69	1
5/23/2022 15:55	273	0
5/23/2022 16:00	284	5

3.3.6. Correlation derivation between features via multivariate time-series regression

Finally, the VAR algorithm was employed to detect anomalies during monitoring, without relying on thresholds. VAR is a multivariate forecasting algorithm used to determine the correlation between two or more time-series. The procedure to build the VAR model involves the following steps [3.3.6.1-3.3.6.6]:

3.3.6.1. Analyzing the characteristics of the time-series. To accurately build the VAR model, the characteristics of each time-series in the dataset should be analyzed. As shown in Fig. 5, which illustrates the actual values of the multi-dimensional time-series used as a test set, a horizontal stationary trend can be observed for all the time-series variables of the first (SS1) and second (SS2) subsystems. Note that there is a strong seasonality.

3.3.6.2. Causality test. The basis of VAR is that each time-series in the dataset influences the others. This means that we can predict the series with its past values along with those of the other series in the dataset. We employed the Granger causality test, which tests the null hypothesis that the past value coefficients in the regression equation are zero. Simply, the past values of the time-series (X) do not cause the other series (Y). Therefore, if the p-value obtained from the test is less than the significance level of 0.05, the null hypothesis can be safely rejected. Johansen's cointegration test (Dwyer, 2015) was employed to establish the presence of a statistically significant relationship between two or more time series.

3.3.6.3. Stationarity test. The VAR model requires the time-series to be predicted to be stationary time-series whose characteristics, such as variance and mean, do not change over time. Therefore, we employed the popular Augmented Dickey-Fuller (ADF) test (Mushtaq, 2011) for this purpose, and utilized the differencing ML technique to convert non-stationary time-series to stationary time-series.

3.3.6.4. Preparing training and test sets. After analyzing the dataset and fulfilling the prerequisites of building a VAR model, the dataset was split into two parts: the training set, which was used to train the model, and the test set, which was used to evaluate the model. We chose the nonrandom sampling method to split the dataset, as this method is typically utilized for time-series, where maintaining the sequence of data is very important. Therefore, 80 % of the dataset was used for training and 20 % for testing.

3.3.6.5. Training the model. The training dataset is utilized to train the VAR model after iteratively fitting increasing orders of the VAR model and picking the order that gives the model with the lowest Akaike information criterion (AIC) statistic (Stock and Watson, 2001).

3.3.6.6. Evaluating the model using the test set. To evaluate the model, we ran it to forecast a dataset that had the same size as the test set and compared the test set against the forecast to compute the accuracy of the model.

4. Results and discussion

In this section, we discuss the results of the ADBP and ADAS components in Sections 4.1 and 4.2, respectively. Section 4.3

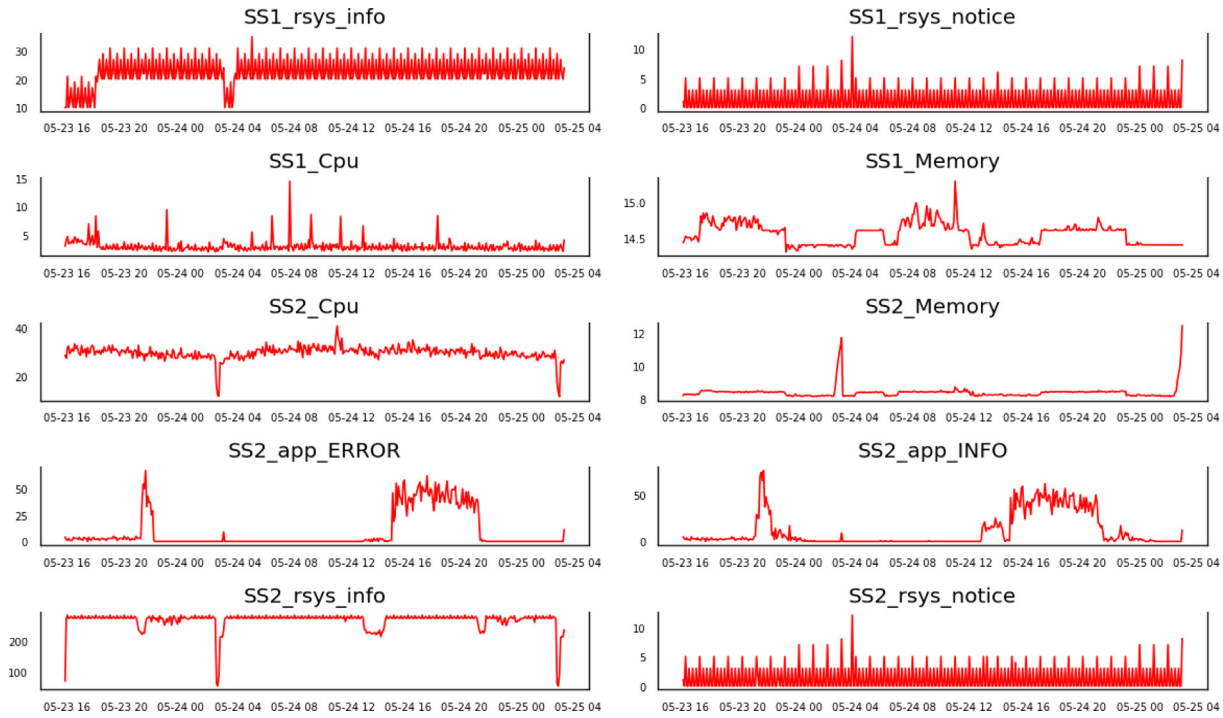

Fig. 5. Actual multi-dimensional time-series for VAR model.

discusses the overall contribution of the proposed **D**evOps **A**nomaly **D**etection **F**ramework (DADF).

4.1. ADBP results and discussion

The experimentation of the ADBP component in the case study of a marketplace web application started in early November 2021 and finished in November 2022. During this period, a dataset describing 25 releases was collected, as listed in Table 3. Fig. 6 shows each release and the values of its corresponding normalized metrics.

First, data and logs concerning software releases were collected for the first 10 successive releases without being processed by the ADBP model to fulfill the requirement of a considerable number of observations required to perform an unsupervised anomaly detection technique. Second, the anomaly detection phase was activated and began to operate in the CI/CD pipeline, as illustrated in Fig. 2. The entry describing the prospective release is evaluated to determine whether it is an anomaly or normal release. Fig. 7 shows the output obtained from the ADBP component after the 25th release.

Finally, to evaluate the ADBP model, we compared the output obtained from the model with the responses received from the

Table 3
ADBP dataset.

ID	Date	Normalized number of lines per commit	Ratio of failed builds	Ratio of failed tests	Ratio of failed deliveries
1	2021-11-01	0.31848	0	0	0
2	2021-11-15	1	0	0	0
3	2021-12-01	0.0392347	0	0	0
4	2021-12-15	0.0344915	0	0.63	0.11
5	2022-01-01	0.0419168	0	0	0
6	2022-01-15	0.0924636	0	0.4	0
7	2022-02-01	0.0103239	0	0	0
8	2022-02-15	0.0088276	0	0	0
9	2022-03-01	0.227597	0	1	0
10	2022-03-15	0.108829	0	0.33	1
11	2022-04-01	0.0157729	0	0	0
12	2022-04-15	0.269608	0	0	0
13	2022-05-01	0.0023151	0	0	0
14	2022-05-15	0.0156694	0	0	0
15	2022-06-01	0.179036	1	0	0
16	2022-06-15	0.135613	0	0	0
17	2022-07-01	0.153974	0	0.17	0.5
18	2022-07-15	0.0044608	0	0	0
19	2022-08-01	0.0021175	0	0	0
20	2022-08-15	0.114053	0	0	0
21	2022-09-01	0.0985714	0	0	0
22	2022-09-15	0.0040468	0	0.5	0
23	2022-10-01	0.0787047	0	0	0
24	2022-10-15	0.10498	0	0	0
25	2022-11-01	0.0757872	0.6	0	0

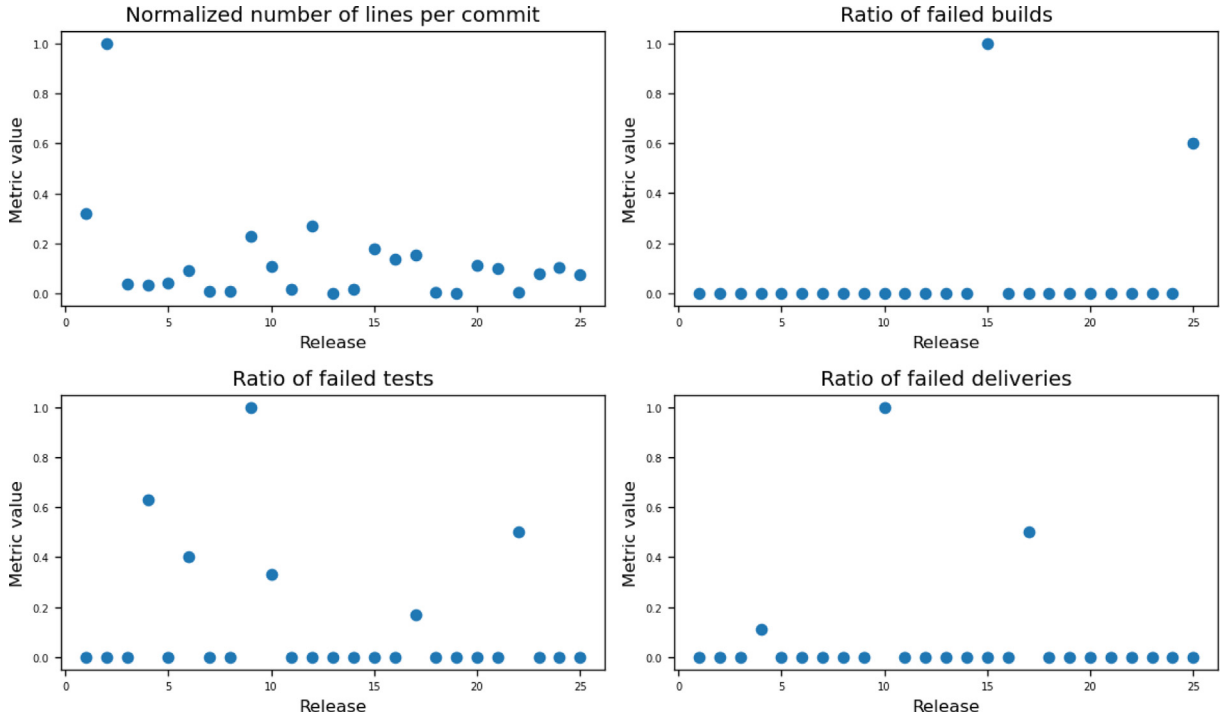

Fig. 6. Set of metrics for each release. (The 25 releases are plotted on the x-axis, and their corresponding metric value is plotted on the y-axis).

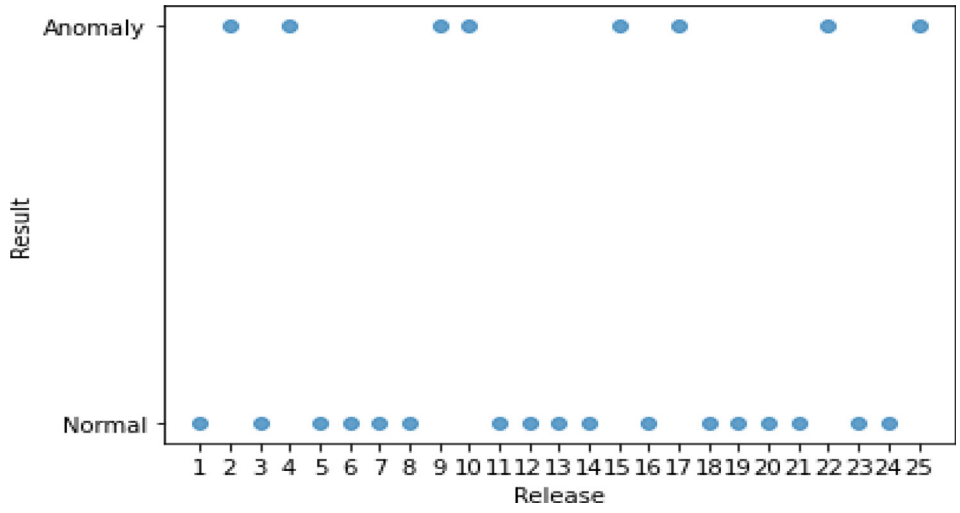

Fig. 7. ADBP output (25 releases plotted on the x-axis and their classification "Normal/Anomaly" on y-axis).

Table 4
Confusion matrix: reported result by the ADBP model based on DevOps team feedback.

	Predicted Anomaly	Predicted Normal	Total
True Anomaly	(TP) 7	(FN) 0	7
True Normal	(FP) 1	(TN) 17	18
Total	(P) 8	(N) 17	25

Table 5
ADBP evaluation metrics based on data from Table 4.

Evaluation Metric	Formula	Score
Accuracy	$\frac{TP+TN}{P+N}$	0.960
Precision	$\frac{TP}{TP+FP}$	0.875
Recall	$\frac{TP}{TP+FN}$	1.000
F1-score	$\frac{2*percision*recall}{percision+recall}$	0.933

DevOps team. Table 4 presents the confusion matrix and the reported total positives (P), total negatives (N), true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). Table 5 presents the results obtained for the ADBP component.

Limitations and future work. The ADBP component was designed to focus only on anomalies. In the event of anomaly detection, alarms are sent to the DevOps team; otherwise, no alarms are sent to keep the team focused solely on anomalies and avoid overburdening them with normal releases. However, the problem of false negatives should be discussed in detail in the future.

Table 6
Dataset used in the experiments.

Collected data	Size
SS1_rsys_log	12,616
SS1_CPU and SS1_Memory	2618
SS2_rsys_log	142,385
SS2_CPU and SS2_Memory	2618
SS2_app_log	8887

Due to the limited literature demonstration for the application of anomaly detection in DevOps, especially in the context of CI/CD. We collected and constructed the dataset from a real-world industrial system that is developed using DevOps practices. However, the size of the collected dataset is still small, as tracking the production of more releases of the same project takes a lot of time. Future research should consider experimenting with a larger dataset.

4.2. ADAS results and discussion

Experimentation of the ADAS component in a case study of a governmental enterprise system serving Egyptian citizens started in May 2022 and finished in July 2022. During this period, we followed the steps described in Section 3.3 and its subsections to:

1. Collect system performance metrics (CPU and memory), application, and system logs. Hence, for the first subsystem (SS1), we collected the system performance metrics and the operating system log (Rsys_log). For the second subsystem (SS2), in addition to collecting the system performance metrics, we collected Rsys_log and application log (app_log). Table 6 lists all the data collected from the two subsystems over 43 successive hours and their corresponding sizes.
2. Parse the collected logs using the drain log parser.
3. Derive other metrics from the parsed logs. Tables 2a, 2b, and 2c show samples of the derived features of the performance metrics, application logs, and system logs, respectively.
4. Aggregate and map the derived features with the performance metrics to construct the dataset.
5. Derive the correlation amongst the aggregated features by analyzing their characteristics and testing their effect on each other.

Table 7
nRMSE for ADAS features.

Feature	nRMSE
SS1_rsys_info	0.0688
SS1_rsys_notice	0.0828
SS1_CPU	0.0285
SS1_Memory	0.1839
SS2_CPU	0.1353
SS2_Memory	0.1309
SS2_app_ERROR	0.6334
SS2_app_INFO	0.5219
SS2_rsys_info	0.1973
SS2_rsys_notice	0.083

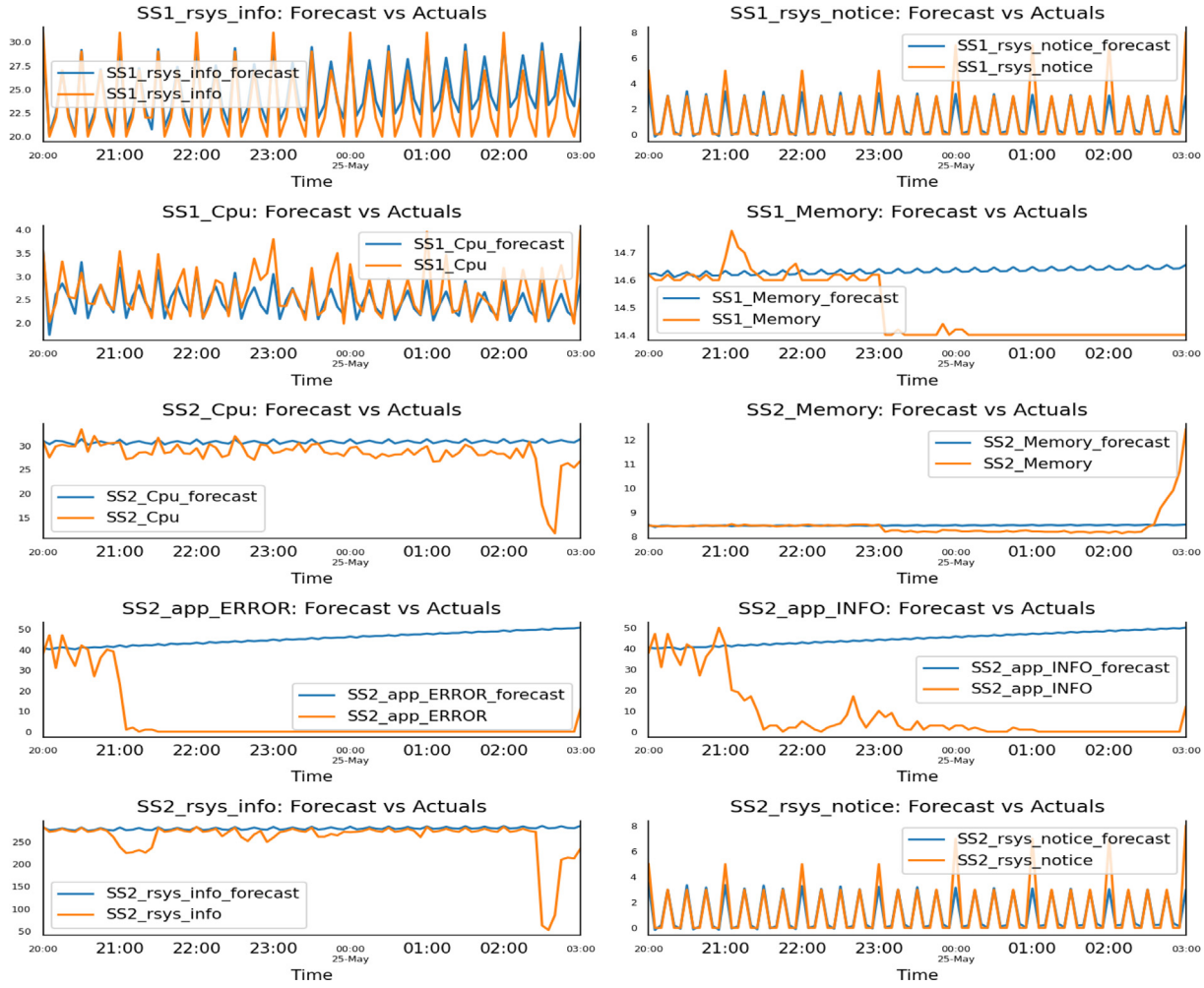

Fig. 8. ADAS forecast versus actuals.

6. Training the ADAS model using the training dataset.

Finally, to evaluate the model, we compared the test set with the forecasted set and calculated the normalized root mean squared error (nRMSE) for each feature, as they have a different range of values. Fig. 8 shows the forecasted set versus the test set (actual values). Table 7 lists the nRMSE for each feature.

Based on the results obtained, as listed in Table 7, we can conclude that the prediction accuracy of most features was relatively high, except for SS2_app_ERROR and SS2_app_INFO, which showed a high error rate. However, these error rates can be considered during baselining by enlarging the range of the forecasted values of each feature with its corresponding nRMSE.

The main goal of the ADAS component is to use ML to establish a baseline for the normal behavior of the monitored system by predicting the values of the performance metrics, application log, and system log for the next window. Then, comparing the baselined values with the actual values to send alarms to the operation team in case of anomaly detection. It is worth noting that while we discuss the ADAS component in the DevOps context, it can also be utilized individually in any other context.

Limitations and future work. Despite the availability of many datasets for log-based anomaly detection, they do not represent the full characteristics of real-world systems. Thus, the ADAS experiments were conducted on a dataset collected and constructed from a real-world industrial system composed of more than one subsystem that is comprised of a software application deployed on a CentOS7 operating system. Future research should consider experimenting with a more complicated system that has more extensive number of features.

The ADAS component utilizes the drain log parser to parse the application and system logs, as it showed superior efficiency based on the evaluation study conducted by (Zhu et al., 2019). However, future research could utilize a different log parser after evaluating other log parsers in terms of time consumption, especially on large-size logs. Whereas, choosing an appropriate log parser can increase the performance and speed of the ADAS component and enable it to operate on a smaller window size.

4.3. *DevOps anomaly detection framework (DADF)*

The main contribution of this research paper is shown in Fig. 1, which illustrates the **D**evOps **A**nomaly **D**etection **F**ramework (DADF) after combining the ADBP and ADAS components. The proposed framework helps DevOps practitioners automatically detect anomalies throughout the lifecycle of DevOps by monitoring all DevOps practices. It also contributes to the research areas of anomaly detection, system monitoring, DevOps, and AIOps, as elaborated in previous sections.

5. Conclusions

In recent years, to improve the reliability and availability of software systems, many studies have been conducted to detect anomalies based on logs by adopting machine learning and artificial intelligence. However, there is scarce literature related to automated anomaly detection in DevOps. Moreover, existing studies still suffer from practical issues because they either rely on a large amount of manually labeled training data (supervised) or unsatisfactory performance (unsupervised and semi-supervised). In this paper, we introduced the **D**evOps **A**nomaly **D**etection **F**ramework (DADF), which is composed of two components that rely on AIOps concepts to automatically detect anomalies throughout the lifecycle of all DevOps practices. The first component is **A**nomaly **D**etection **B**efore **P**roduction (ADBP), which aims to detect anomalies early before deploying the prospective release into production to mitigate production problems. The second component is **A**nomaly **D**etection **A**fter **S**taging (ADAS), which intends to detect anomalies during system monitoring by establishing a baseline for the normal behavior of the monitored system by predicting the values of the performance metrics, application log, and system log. Then, without relying on fixed rules or thresholds, the baselined values are compared with the actual values to send alarms to the operation team in the case of anomaly detection. We experimentally evaluated the ADBP and ADAS components separately in two real-world industrial projects. The experimental results demonstrated the effectiveness of DADF in detecting anomalies that arise during the DevOps lifecycle.

Financial Disclosure

None reported.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Alibi, M., Roy, B., 2016. Mastering CentOS 7 Linux Server. Packt Publishing Ltd.
Allen Jones, 2022. DSP model [WWW Document]. URL https://www.itprotoday.com/devops-and-software-development/development-staging-and-production-model (accessed 12.31.22).
Anderson, C., 2015. Docker [software engineering]. IEEE Softw. 32, 102–c3.
Bass, L., 2017. The software architect and DevOps. IEEE Softw. 35, 8–10.
Battina, D.S., 2019. An intelligent devops platform research and design based on machine learning. Training 6.
Bosch, N., Bosch, J., 2020. Software Logs for Machine Learning in a DevOps Environment. In: 2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). pp. 29–33.
Breunig, M.M., Kriegel, H.-P., Ng, R.T., Sander, J., 2000. LOF: identifying density-based local outliers, in: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. pp. 93–104.
Cândido, J., Aniche, M., van Deursen, A., 2021. Log-based software monitoring: a systematic mapping study. PeerJ Comput. Sci. 7, e489.
Capizzi, A., Distefano, S., Araújo, L.J.P., Mazzara, M., Ahmad, M., Bobrov, E., 2019. Anomaly detection in DevOps toolchain. In: International Workshop on Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment. pp. 37–51.
Dang, Y., Lin, Q., Huang, P., 2019. AIOps: real-world challenges and research innovations. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). pp. 4–5.
Du, M., Li, F., Zheng, G., Srikumar, V., 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 1285–1298.
DuBois, P., 2008. MySQL. Pearson Education.
Dwyer, G.P., 2015. The Johansen tests for cointegration. White Paper 1–7.
Ebert, C., Gallardo, G., Hernantes, J., Serrano, N., 2016. DevOps. IEEE Softw. 33, 94–100.
Farshchi, M., Schneider, J.-G., Weber, I., Grundy, J., 2018. Metric selection and anomaly detection for cloud operations using log and metric correlation analysis. J. Syst. Softw. 137, 531–549.
GitLab [WWW Document], 2022. URL https://docs.gitlab.com/ee/api/ (accessed 12.31.22).
He, P., Zhu, J., He, S., Li, J., Lyu, M.R., 2016. An evaluation study on log parsing and its use in log mining. In: 2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 654–661.
He, S., Lin, Q., Lou, J.-G., Zhang, H., Lyu, M.R., Zhang, D., 2018. Identifying impactful service system problems via log analysis. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 60–70.
Huch, F., Golagha, M., Petrovska, A., Krauss, A., 2018. Machine learning-based runtime anomaly detection in software systems: An industrial evaluation. In: 2018 IEEE Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE). pp. 13–18.
Islam, M.S., Pourmajidi, W., Zhang, L., Steinbacher, J., Erwin, T., Miranskyy, A., 2021. Anomaly detection in a large-scale cloud platform. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). pp. 150–159.

Khatuya, S., Ganguly, N., Basak, J., Bharde, M., Mitra, B., 2018. Adele: Anomaly detection from event log empiricism. In: IEEE INFOCOM 2018-IEEE Conference on Computer Communications. pp. 2114–2122.

Le, V.-H., Zhang, H., 2022. Log-based anomaly detection with deep learning: How far are we?. In: Proceedings of the 44th International Conference on Software Engineering. pp. 1356–1367.

Li, Y., Jiang, Z.M., Li, H., Hassan, A.E., He, C., Huang, R., Zeng, Z., Wang, M., Chen, P., 2020. Predicting node failures in an ultra-large-scale cloud computing platform: an aiops solution. ACM Trans. Softw. Eng. Methodol. (TOSEM) 29, 1–24.

Lou, J.-G., Fu, Q., Yang, S., Xu, Y., Li, J., 2010. Mining invariants from console logs for system problem detection. In: 2010 USENIX Annual Technical Conference (USENIX ATC 10).

Mushtaq, R., 2011. Augmented dickey fuller test. World J. Finance Investment Res. 1.

Paper, D., Paper, D., 2020. Introduction to scikit-learn. Hands-on Scikit-Learn for Machine Learning Applications: Data Science Fundamentals with Python 1–35.

Pourmajidi, W., Steinbacher, J., Erwin, T., Miranskyy, A., 2018. On challenges of cloud monitoring. arXiv preprint arXiv:1806.05914.

Rowse, M., Cohen, J., 2021. A survey of DevOps in the south African software context. In: Proceedings of the 54th Hawaii International Conference on System Sciences. p. 6785.

Shahin, M., Babar, M.A., Zahedi, M., Zhu, L., 2017. Beyond continuous delivery: an empirical investigation of continuous deployment challenges. In: 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). pp. 111–120.

Stauffer, M., 2019. Laravel: Up & Running: A Framework for Building Modern php apps. O'Reilly Media.

Stock, J.H., Watson, M.W., 2001. Vector autoregressions. J. Econ. Perspect. 15, 101–115.

Sun, D., Fu, M., Zhu, L., Li, G., Lu, Q., 2016. Non-intrusive anomaly detection with streaming performance metrics and logs for DevOps in public clouds: a case study in AWS. IEEE Trans. Emerg. Top. Comput. 4, 278–289.

Yang, L., Chen, J., Wang, Z., Wang, W., Jiang, J., Dong, X., Zhang, W., 2021. Semi-supervised log-based anomaly detection via probabilistic label estimation. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). pp. 1448–1460.

Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., others, 2019. Robust log-based anomaly detection on unstable log data, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 807–817.

Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z., Lyu, M.R., 2019. Tools and benchmarks for automated log parsing. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). pp. 121–130.